GRU-RNN & GRU-ODE-Bayes & Continuous Normalizing Flows

Machine Learning project for life sciences

Lorenzo Spiridioni & Maxim Hirschmann

February 10, 2025

Table of contents

Our starting point

Introduction

- **JAX newcomers**: We were starting from scratch here. A big part of our motivation for this project was simply wanting to get our hands on the framework and see what the hype is about.
- **Moving beyond the ML basics**: We've got the fundamentals down, but we wanted to dive deeper into such an important topic in current times

Table of contents

2 GRU-ODE-Bayes

Core Concepts

GRU-ODE-Bayes

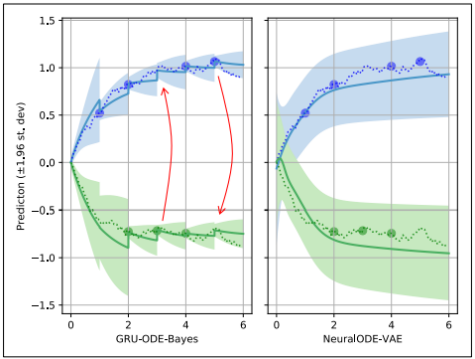


Figure: Illustration of continuous prediction + discrete Bayesian updates (paper Fig.1).

Core Concepts

GRU-ODE-Bayes

- **High-level idea:** alternate two modes
  - **Predictor (continuous):** evolve a latent hidden state in continuous time using an ODE-inspired GRU (GRU-ODE).

Core Concepts

GRU-ODE-Bayes

- **High-level idea:** alternate two modes
  - **Predictor (continuous):** evolve a latent hidden state in continuous time using an ODE-inspired GRU (GRU-ODE).
  - **Corrector (discrete):** when an observation arrives, perform a Bayesian-style update of the hidden state (GRU-Bayes).

Core Concepts

GRU-ODE-Bayes

- **High-level idea:** alternate two modes
  - **Predictor (continuous):** evolve a latent hidden state in continuous time using an ODE-inspired GRU (GRU-ODE).
  - **Corrector (discrete):** when an observation arrives, perform a Bayesian-style update of the hidden state (GRU-Bayes).
- **Why this matters:**
  - Models time continuously → naturally handles irregular timestamps.
  - Updates only when data arrives → natively handles asynchronous, feature-level missingness.
  - Learns uncertainty evolution + reduces variance upon observations (cf. figure).

Formulation

GRU-ODE-Bayes

- GRU hidden update (discrete-time) decomposes into:

$$
\begin{aligned}
r_t &= \sigma(W_r x_t + U_r h_{t-1} + b_r) \quad \text{(reset gate)} \\
z_t &= \sigma(W_z x_t + U_z h_{t-1} + b_z) \quad \text{(update gate)} \\
g_t &= \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h) \\
h_t &= z_t \odot h_{t-1} + (1 - z_t) \odot g_t
\end{aligned}
$$

Formulation

GRU-ODE-Bayes

- GRU hidden update (discrete-time) decomposes into:

$$
\begin{aligned}
r_t &= \sigma(W_r x_t + U_r h_{t-1} + b_r) \quad \text{(reset gate)} \\
z_t &= \sigma(W_z x_t + U_z h_{t-1} + b_z) \quad \text{(update gate)} \\
g_t &= \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h) \\
h_t &= z_t \odot h_{t-1} + (1 - z_t) \odot g_t
\end{aligned}
$$

- **Reset gate (r_t):** controls how much past hidden state to forget. Intuition: when r_t is small $\rightarrow$ forget old memory for the candidate.

Formulation

GRU-ODE-Bayes

- GRU hidden update (discrete-time) decomposes into:

$$
\begin{aligned}
r_t &= \sigma(W_r x_t + U_r h_{t-1} + b_r) \quad \text{(reset gate)} \\
z_t &= \sigma(W_z x_t + U_z h_{t-1} + b_z) \quad \text{(update gate)} \\
g_t &= \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h) \\
h_t &= z_t \odot h_{t-1} + (1 - z_t) \odot g_t
\end{aligned}
$$

- **Reset gate (r_t):** controls how much past hidden state to forget. Intuition: when r_t is small $\rightarrow$ forget old memory for the candidate.
- **Update gate (z_t):** interpolates between keeping the old state h_{t-1} vs. adopting the new candidate g_t.

Formulation

GRU-ODE-Bayes

- Start from the GRU difference form and take a continuous-time limit to obtain an ODE for the hidden state:

$$\frac{dh(t)}{dt} = (1 - z(t)) \odot (g(t) - h(t)).$$

Formulation

GRU-ODE-Bayes

- Start from the GRU difference form and take a continuous-time limit to obtain an ODE for the hidden state:

$$\frac{dh(t)}{dt} = (1 - z(t)) \odot (g(t) - h(t)).$$

- GRU-Bayes processes incoming sporadic observations and updates the latent h discretely:

$$h(t^+) = \mathrm{GRU}\big(h(t^-),\ f_{\text{prep}}(y, m, h(t^-))\big).$$

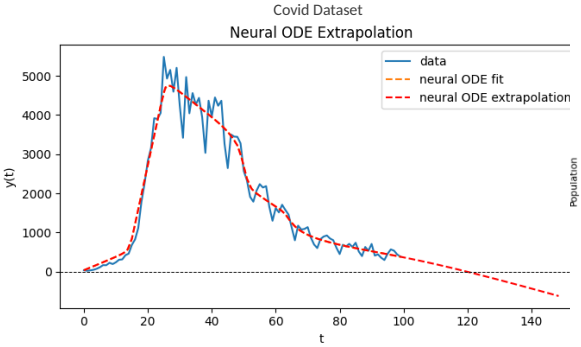
- f_{prep} merges the observed values, masks and current predictions into an input suitable for the GRU update.

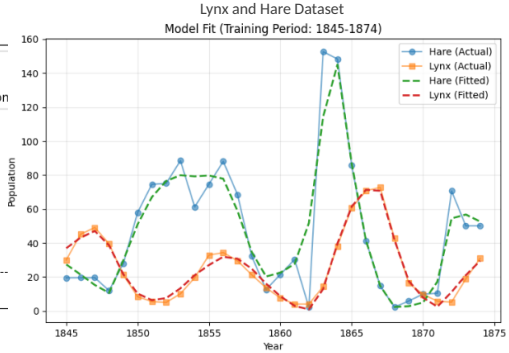
Table of contents

3 GRU-ODE-Bayes Implementations and Results

Lynx and Hare

GRU-ODE-Bayes Implementations and Results





dataset	method	training loss
covid cases	simple neural ode	0.06
lynx-and-hare	simple neural ode	0.08

Spirals

GRU-ODE-Bayes Implementations and Results

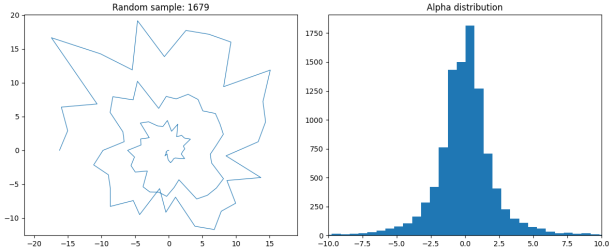
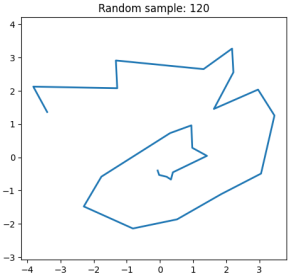


Table: Comparison of Test Loss across ODE-method

Method	Test Loss
GRU-ODE	0.0266
MLP	0.0017
ODE-RNN	0.0120

Spirals - Irregular and missing data

GRU-ODE-Bayes Implementations and Results



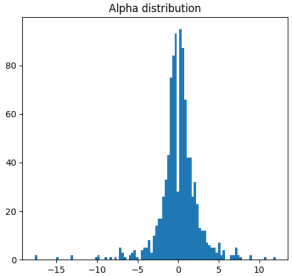
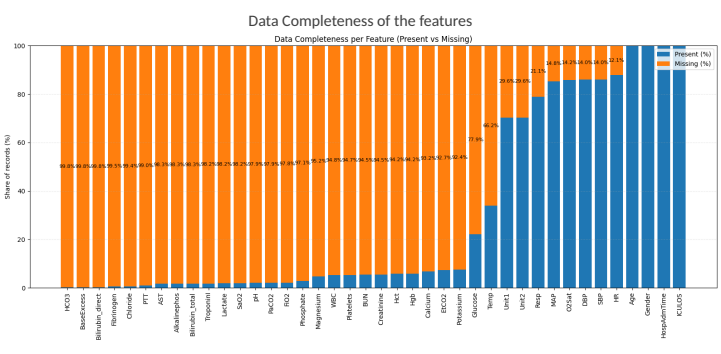


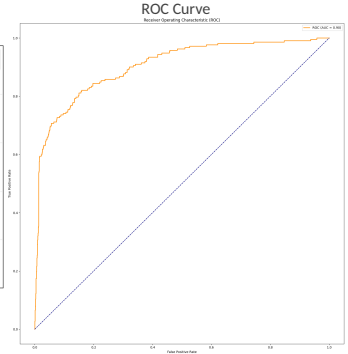
Table: ODE-RNN Performance vs. Data Availability (Spiral Dataset)

% Data Available	Loss (α prediction)
100%	0.0061
75%	0.0097
50%	0.0275
25%	0.0559

Sepsis

GRU-ODE-Bayes Implementations and Results





Open Source Contribution

GRU-ODE-Bayes Implementations and Results

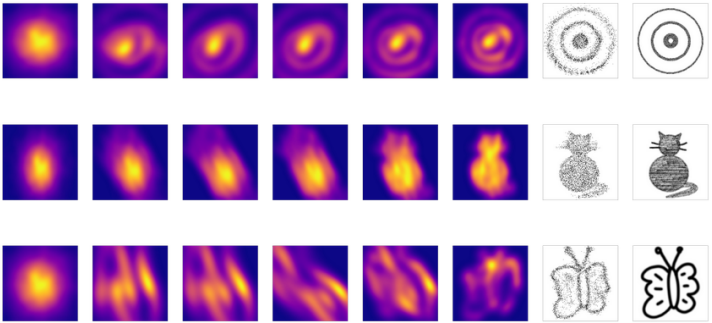
Figure: https://github.com/patrick-kidger/diffrax/pull/728

Table of contents

4 Continuous Normalizing Flows

Core Concepts

Continuous Normalizing Flows



- **Flowing Toward the Target Density:** Each intermediate distribution is a small step of the flow, gradually reshaping probability mass until it matches the target distribution (gray) at the final time.

Formulation

Continuous Normalizing Flows

- Start with a base distribution (Usually Gaussian): $z \sim p_z(z)$ (often $\mathcal{N}(0, I)$)

Formulation

Continuous Normalizing Flows

- Start with a base distribution (Usually Gaussian): $z \sim p_z(z)$ (often $\mathcal{N}(0, I)$)
- Define an invertible transformation:

$$\frac{dy(t)}{dt} = f_\theta(t, y(t))$$

where $y(t)$ evolves over time t from the base distribution to the data distribution, and f_θ is a neural network parameterized by θ.

Formulation

Continuous Normalizing Flows

- Start with a base distribution (Usually Gaussian): $z \sim p_z(z)$ (often $\mathcal{N}(0, I)$)
- Define an invertible transformation:

$$\frac{dy(t)}{dt} = f_\theta(t, y(t))$$

  where $y(t)$ evolves over time t from the base distribution to the data distribution, and f_θ is a neural network parameterized by θ.
- This framework allows us to generate samples by integrating the ODE forward in time, and to evaluate exact (or unbiased approximate) log-likelihoods by integrating a coupled ODE for the log-density.

Formulation

Continuous Normalizing Flows

- Let the state evolve according to a neural ODE: $\frac{dy(t)}{dt} = f_\theta(t, y(t))$
- The log-density evolves via the instantaneous change-of-variables rule:

$$\frac{d}{dt} \log p(y(t)) = -\operatorname{tr}\left(\frac{\partial f_\theta(t, y(t))}{\partial y}\right)$$

(In practice, the Jacobian trace is computed efficiently using Hutchinson's stochastic estimator.)

Formulation

Continuous Normalizing Flows

- Let the state evolve according to a neural ODE: $\frac{dy(t)}{dt} = f_\theta(t, y(t))$
- The log-density evolves via the instantaneous change-of-variables rule:

$$\frac{d}{dt} \log p(y(t)) = -\operatorname{tr}\left(\frac{\partial f_\theta(t, y(t))}{\partial y}\right)$$

(In practice, the Jacobian trace is computed efficiently using Hutchinson's stochastic estimator.)

- To evaluate the likelihood of a data point x, we integrate backwards in time:

$$\log p(x) = \log p(z) + \int_{t_1}^{t_0} \operatorname{tr}\left(\frac{\partial f_\theta}{\partial y}\right) dt, \quad z = y(t_0) \sim \mathcal{N}(0, I)$$

Formulation

Continuous Normalizing Flows

- Let the state evolve according to a neural ODE: $\frac{dy(t)}{dt} = f_\theta(t, y(t))$
- The log-density evolves via the instantaneous change-of-variables rule:

$$\frac{d}{dt} \log p(y(t)) = -\operatorname{tr}\left(\frac{\partial f_\theta(t, y(t))}{\partial y}\right)$$

(In practice, the Jacobian trace is computed efficiently using Hutchinson's stochastic estimator.)

- To evaluate the likelihood of a data point x, we integrate backwards in time:

$$\log p(x) = \log p(z) + \int_{t_1}^{t_0} \operatorname{tr}\left(\frac{\partial f_\theta}{\partial y}\right) dt, \quad z = y(t_0) \sim \mathcal{N}(0, I)$$

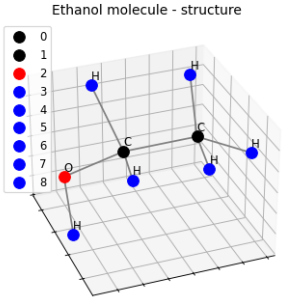
- Training objective (maximum likelihood):

$$\max_\theta \ \mathbb{E}_{x \sim p_{\text{data}}} [\log p_\theta(x)] \quad \Leftrightarrow \quad \min_\theta -\frac{1}{N} \sum_{i=1}^{N} w_i \log p_\theta(x_i)$$

Table of contents

5 Continuous Normalizing Flows Implementations and Results

Ethanol sampling

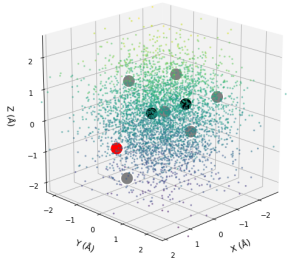


- 5,000 structural variations of the ethanol molecule, each a matrix of size 9×3 (9 atoms, 3 dimensions)
- Try to learn the total distribution of the atoms to sample new datapoints for future uses (sampling / generative AI)

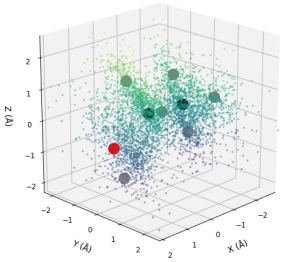
Ethanol sampling results

5 Continuous Normalizing Flows Implementations and Results

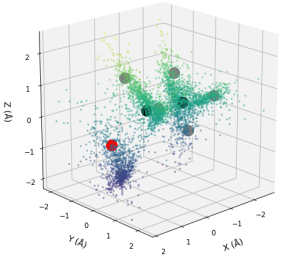
Step 0: Base Distribution (Prior Noise)



Step 2: Intermediate Flow (t = 0.67)



Step 3: Learned Distribution



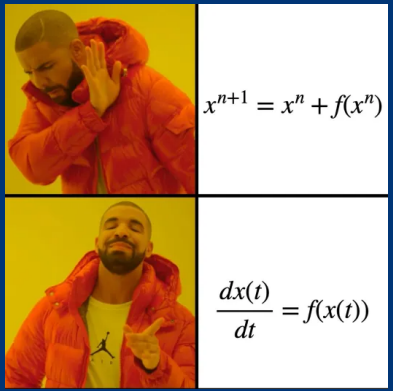
Likelihood Estimation

Continuous Normalizing Flows Implementations and Results

Image	Total log-likelihood	Avg log-likelihood/point
cat.png	-173066.125	-26.922016
cat-eval.jpg	-548387.500	-26.911974

GRU-RNN & GRU-ODE-Bayes & Continuous Normalizing Flows

Thank you for your attention!



figures/qr-code.jpg